

Study of Digital Modulation Techniques: BER vs SNR Performance and Constellation Analysis

(M-PAM, M-QAM and M-PSK over an AWGN Channel)

Advanced Communication Laboratory

Name: Harsh Kumar

Roll No: 226EC5003

1. Aim

To generate and compare the theoretical Bit Error Rate (BER) versus Signal-to-Noise Ratio (E_b/N_0) performance curves, along with the signal-space constellation diagrams, for the following digital modulation schemes transmitted over an AWGN (Additive White Gaussian Noise) channel:

- Pulse Amplitude Modulation (PAM): 2-PAM, 4-PAM, 8-PAM, 16-PAM
- Quadrature Amplitude Modulation (QAM): 4-QAM, 8-QAM, 16-QAM, 64-QAM
- Phase Shift Keying (PSK): BPSK, QPSK, 8-PSK, 16-PSK

and to analyse how the order (M) of each modulation scheme affects error performance and spectral/power efficiency trade-offs.

2. Software Used

- Python 3 (numpy, scipy, matplotlib)
- `scipy.special.erfc` — used to evaluate the Gaussian Q-function
- Text editor / Jupyter or any Python IDE for execution
- (Equivalent results can also be reproduced in MATLAB using `berawgn/qfunc` and the Communications Toolbox.)

3. Theory

3.1 Digital Modulation and the AWGN Channel

In digital communication, a stream of information bits is mapped to one of M discrete symbols drawn from a signal constellation before transmission. When the channel corrupts the transmitted signal with Additive White Gaussian Noise (AWGN), the receiver may decide in favour of the wrong symbol, producing bit errors. The probability of such an error — the Bit Error Rate (BER) — depends on the constellation geometry (minimum distance between points), the number of points M , the bit-to-symbol mapping (Gray coding is assumed throughout), and the ratio of energy per bit to noise power spectral density, E_b/N_0 .

The Gaussian Q-function, used throughout this report, is defined as:

$$Q(x) = 0.5 \cdot \operatorname{erfc}(x / \sqrt{2})$$

3.2 M-ary PAM

In M-PAM, information is carried purely in the amplitude of a single carrier axis; the M constellation points are equally spaced real values symmetric about the origin. Because points at the edges of the constellation have only one neighbour, while inner points have two, the average symbol error probability for Gray-coded M-PAM over AWGN is:

$$P_b = [2(M-1) / (M \cdot \log_2 M)] \cdot Q(\sqrt{ 6 \cdot \log_2 M / (M^2-1) } \cdot \sqrt{E_b/N_0})$$

As M increases, symbols are packed closer together for the same average energy, so the minimum distance decreases and BER rises for a fixed E_b/N_0 — i.e., higher-order PAM trades noise robustness for higher bit rate (more bits/symbol).

3.3 M-ary QAM

QAM modulates both the amplitude and phase of the carrier by combining two independent PAM signals on orthogonal in-phase (I) and quadrature (Q) axes, giving a two-dimensional constellation. For square constellations ($M = 4, 16, 64$, i.e. $M = 2^{\text{even}}$), the Gray-coded bit-error probability is well approximated by:

$$P_b \approx (4/\log_2 M) \cdot (1 - 1/\sqrt{M}) \cdot Q(\sqrt{ 3 \cdot \log_2 M / (M-1) } \cdot \sqrt{E_b/N_0})$$

8-QAM cannot form a perfect square grid, so a rectangular/'cross' constellation (two amplitude levels $\times$ four phases) is used instead; its exact BER differs slightly from the square-QAM formula, which is nonetheless used here as a standard approximation. Because QAM spreads points over two dimensions instead of one, it achieves a better power efficiency than PAM of the same order M .

3.4 M-ary PSK

In M-PSK, all constellation points lie on a circle of constant amplitude and are distinguished only by phase, spaced $2\pi/M$ apart. The Gray-coded bit-error probability is exactly known for BPSK and (per-bit) for QPSK, and closely approximated for higher orders by:

$$\text{BPSK / QPSK: } P_b = Q(\sqrt{2 \cdot E_b/N_0})$$

$$\text{M-PSK (} M \geq 8 \text{): } P_b \approx (2/\log_2 M) \cdot Q(\sqrt{2 \cdot \log_2 M} \cdot \sin(\pi/M) } \cdot \sqrt{E_b/N_0})$$

Since QPSK can be viewed as two independent, orthogonal BPSK signals (on the I and Q rails), its per-bit BER is identical to BPSK, even though it carries twice the data rate for the same bandwidth — this is why the BPSK and QPSK curves overlap exactly in Section 6. As M grows beyond 4, points crowd angularly, the effective distance between neighbours shrinks for a fixed energy, and BER increases sharply — M-PSK therefore degrades faster with M than M-QAM of the same order.

3.5 Constellation Diagrams

A constellation diagram plots each symbol as a point in the complex I-Q plane. It visually captures three things relevant to performance: (a) the number of points M , (b) the minimum Euclidean distance between neighbouring points (which governs noise immunity), and (c) whether energy is spent on amplitude, phase, or both. All constellations in this report are normalised to unit average symbol energy so that different schemes can be compared fairly on the same E_b/N_0 axis.

4. Source Code

The following Python script computes the theoretical BER expressions above, generates the normalised constellation points for each scheme, and plots/saves all figures used in this report.

```
"""
Generate BER vs SNR (Eb/N0) plots and constellation diagrams for:
1) M-PAM : 2, 4, 8, 16
2) M-QAM : 4, 8, 16, 64
3) M-PSK : BPSK, QPSK, 8-PSK, 16-PSK

Theoretical (analytical) BER expressions over an AWGN channel,
assuming Gray-coded bit-to-symbol mapping.
"""

import os
import numpy as np
import matplotlib.pyplot as plt
from scipy.special import erfc

plt.rcParams.update({
    "figure.dpi": 140,
```

```

    "font.size": 11,
    "axes.grid": True,
    "grid.alpha": 0.35,
})

os.makedirs("figs", exist_ok=True)

def Q(x):
    """Gaussian Q-function."""
    return 0.5 * erfc(x / np.sqrt(2))

EbN0_dB = np.arange(-2, 22, 1)
EbN0 = 10 ** (EbN0_dB / 10)

# -----
# 1. THEORETICAL BER FORMULAE
# -----

def ber_mpam(M, EbN0):
    k = np.log2(M)
    return (2 * (M - 1)) / (M * k) * Q(np.sqrt((6 * k / (M**2 - 1)) * EbN0))

def ber_mqam(M, EbN0):
    k = np.log2(M)
    return (4 / k) * (1 - 1 / np.sqrt(M)) * Q(np.sqrt((3 * k / (M - 1)) * EbN0))

def ber_mpsk(M, EbN0):
    k = np.log2(M)
    if M == 2:
        return Q(np.sqrt(2 * EbN0))
    if M == 4:
        return Q(np.sqrt(2 * EbN0))
    return (2 / k) * Q(np.sqrt(2 * k * EbN0)) * np.sin(np.pi / M)

# -----
# 2. CONSTELLATION POINT GENERATORS
# -----

def const_mpam(M):
    levels = np.arange(-(M - 1), M, 2)
    levels = levels / np.sqrt(np.mean(levels ** 2)) # normalize avg energy = 1
    return levels + 0j

def const_mqam(M):
    """Square QAM for 4, 16, 64; rectangular/cross layout for 8-QAM."""
    if M in (4, 16, 64):
        side = int(np.sqrt(M))
        levels = np.arange(-(side - 1), side, 2)
        I, Qd = np.meshgrid(levels, levels)
        pts = (I + 1j * Qd).flatten()

    elif M == 8:
        levels_i = np.array([-3, -1, 1, 3])
        levels_q = np.array([-1, 1])
        I, Qd = np.meshgrid(levels_i, levels_q)
        pts = (I + 1j * Qd).flatten()

    else:
        raise ValueError("Unsupported M")

    pts = pts / np.sqrt(np.mean(np.abs(pts) ** 2))

```

```

return pts

def const_mpsk(M):
    m = np.arange(M)
    phase = 2 * np.pi * m / M
    return np.exp(1j * phase)

# -----
# 3. PLOTTING HELPERS
# -----

def plot_ber(family_name, M_list, ber_func, fname, colors):
    plt.figure(figsize=(7, 5.5))

    for M, c in zip(M_list, colors):
        ber = ber_func(M, EbN0)

        if family_name == "PSK":
            label = {2: "BPSK", 4: "QPSK"}.get(M, f"{M}-PSK")
        else:
            label = f"{M}-{family_name}"

        plt.semilogy(EbN0_dB, ber, marker="o", markersize=4,
                     linewidth=1.8, color=c, label=label)

    plt.ylim(1e-6, 1)
    plt.xlim(EbN0_dB[0], EbN0_dB[-1])
    plt.xlabel("Eb/N0 (dB)")
    plt.ylabel("Bit Error Rate (BER)")
    plt.title(f"BER vs SNR (Eb/N0) - {family_name} family")
    plt.legend()
    plt.grid(True, which="both", linestyle="--")
    plt.tight_layout()
    plt.savefig(fname, dpi=150)
    plt.show()
    plt.close()

def plot_constellations(family_name, M_list, const_func, fname, names=None):
    fig, axes = plt.subplots(1, len(M_list), figsize=(4 * len(M_list), 4.2))
    if len(M_list) == 1:
        axes = [axes]

    for ax, M, nm in zip(axes, M_list, names):
        pts = const_func(M)

        ax.scatter(pts.real, pts.imag, s=90, color="#2563eb",
                  zorder=3, edgecolor="k")
        ax.axhline(0, color="gray", linewidth=0.8)
        ax.axvline(0, color="gray", linewidth=0.8)

        lim = max(1.6, np.max(np.abs(pts)) * 1.4)
        ax.set_xlim(-lim, lim)
        ax.set_ylim(-lim, lim)
        ax.set_aspect("equal")
        ax.set_title(nm)
        ax.set_xlabel("In-phase (I)")
        ax.set_ylabel("Quadrature (Q)")
        ax.grid(True, linestyle="--", alpha=0.5)

    fig.suptitle(f"Constellation Diagrams - {family_name} family", y=1.03, fontsize=13)
    plt.tight_layout()
    plt.savefig(fname, bbox_inches="tight", dpi=150)
    plt.show()
    plt.close()

```

```

# -----
# 4. GENERATE ALL PLOTS
# -----

colors4 = ["#1d4ed8", "#059669", "#d97706", "#dc2626"]

# ---- PAM ----
pam_M = [2, 4, 8, 16]
plot_ber("PAM", pam_M, ber_mpam, "figs/ber_pam.png", colors4)
plot_constellations("PAM", pam_M, const_mpam, "figs/const_pam.png",
                    names=["2-PAM", "4-PAM", "8-PAM", "16-PAM"])

# ---- QAM ----
qam_M = [4, 8, 16, 64]
plot_ber("QAM", qam_M, ber_mqam, "figs/ber_qam.png", colors4)
plot_constellations("QAM", qam_M, const_mqam, "figs/const_qam.png",
                    names=["4-QAM", "8-QAM", "16-QAM", "64-QAM"])

# ---- PSK ----
psk_M = [2, 4, 8, 16]
psk_names = ["BPSK", "QPSK", "8-PSK", "16-PSK"]
plot_ber("PSK", psk_M, ber_mpsk, "figs/ber_psk.png", colors4)
plot_constellations("PSK", psk_M, const_mpsk, "figs/const_psk.png", names=psk_names)

# ---- Cross-family comparison at M = 4 ----
plt.figure(figsize=(7.5, 6))
plt.semilogy(EbN0_dB, ber_mpam(4, EbN0), "-o", ms=4, label="4-PAM")
plt.semilogy(EbN0_dB, ber_mqam(4, EbN0), "-s", ms=4, label="4-QAM")
plt.semilogy(EbN0_dB, ber_mpsk(4, EbN0), "-^", ms=4, label="QPSK (4-PSK)")
plt.ylim(1e-6, 1)
plt.xlabel("Eb/N0 (dB)")
plt.ylabel("BER")
plt.title("Cross-family comparison at M = 4")
plt.legend()
plt.grid(True, which="both", linestyle="--")
plt.tight_layout()
plt.savefig("figs/ber_cross_family_M4.png", dpi=150)
plt.show()
plt.close()

print("All figures generated successfully in ./figs")

```

5. Procedure

1. Import numpy, scipy.special.erfc and matplotlib; define the Gaussian Q-function $Q(x) = 0.5 \cdot \text{erfc}(x/\sqrt{2})$.
2. Define the E_b/N_0 sweep range in dB (−2 dB to 21 dB) and convert to a linear scale.
3. Implement the closed-form/approximate theoretical BER equations for M-PAM, M-QAM and M-PSK as functions of M and E_b/N_0 .
4. Implement generator functions that return the normalised complex constellation points for each modulation order (PAM on the real axis, square/cross grids for QAM, unit-circle points for PSK).
5. For each family (PAM, QAM, PSK), loop over the required orders M, compute the BER curve, and plot all curves together on a single semi-log (log10 y-axis) graph of BER vs E_b/N_0 .
6. For the same family, plot the corresponding constellation diagrams for every M side-by-side for visual comparison.
7. Save each figure as a PNG file and repeat for all three families; also generate one cross-family comparison plot at $M = 4$ for additional insight.
8. Interpret the resulting curves and diagrams to draw conclusions about power efficiency versus spectral efficiency trade-offs (Section 7).

6. Observations

The figures below were generated directly by the script in Section 4.

6.1 M-PAM: Constellations and BER

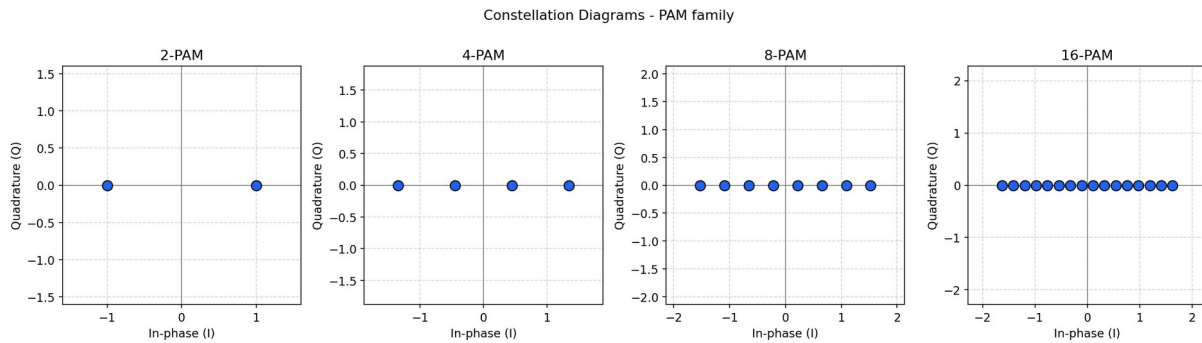


Fig. 6.1(a): Constellation diagrams — 2/4/8/16-PAM (normalised unit energy)

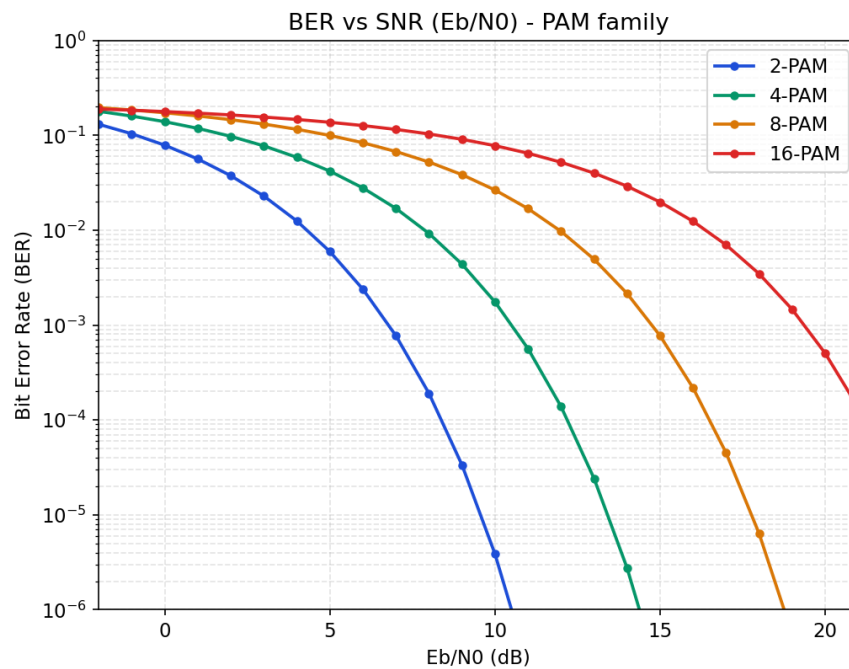


Fig. 6.1(b): BER vs E_b/N_0 comparison for the PAM family

6.2 M-QAM: Constellations and BER

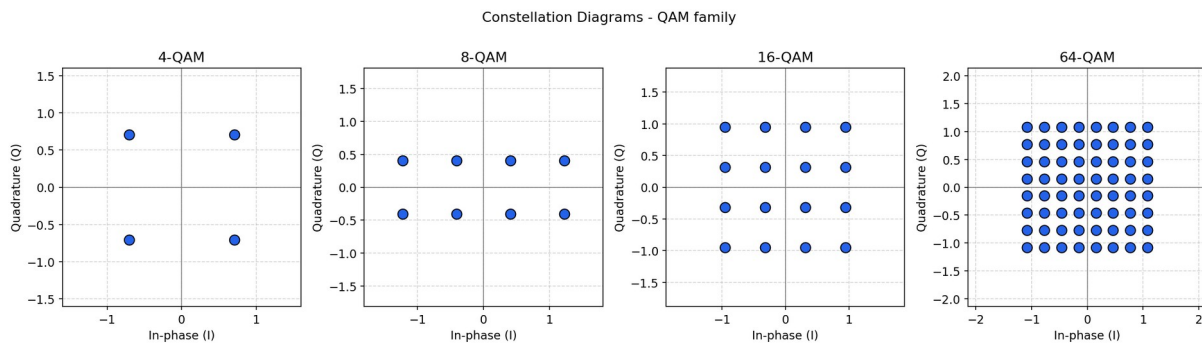


Fig. 6.2(a): Constellation diagrams — 4/8/16/64-QAM (8-QAM uses a cross layout)

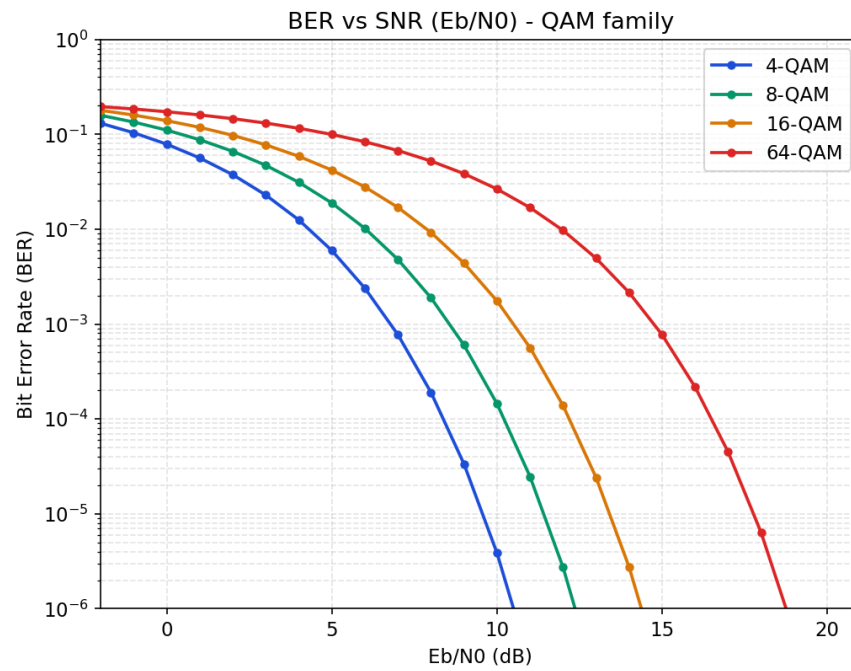


Fig. 6.2(b): BER vs E_b/N_0 comparison for the QAM family

6.3 M-PSK: Constellations and BER

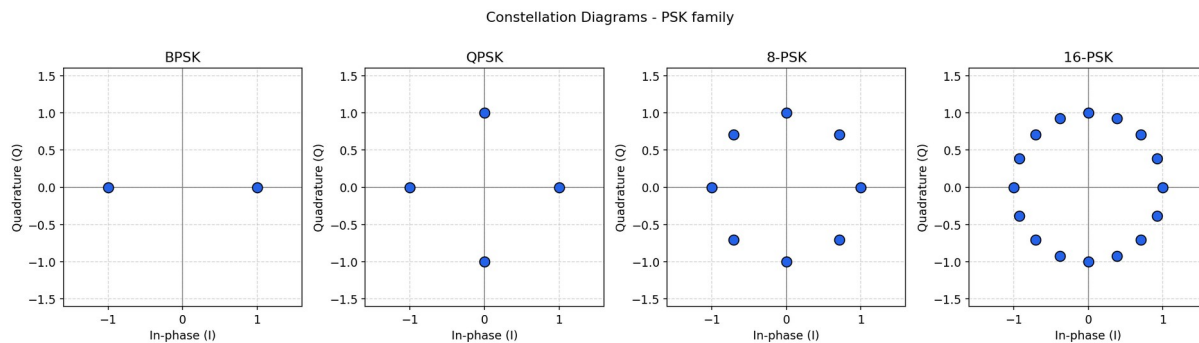


Fig. 6.3(a): Constellation diagrams — BPSK/QPSK/8-PSK/16-PSK (unit circle)

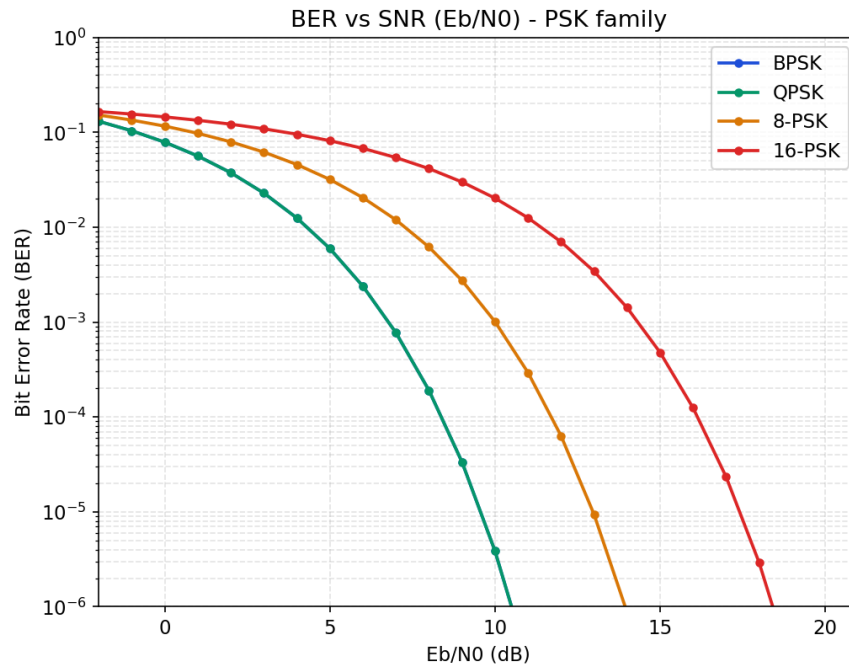


Fig. 6.3(b): BER vs E_b/N_0 comparison for the PSK family

6.4 Cross-family Comparison ($M = 4$)

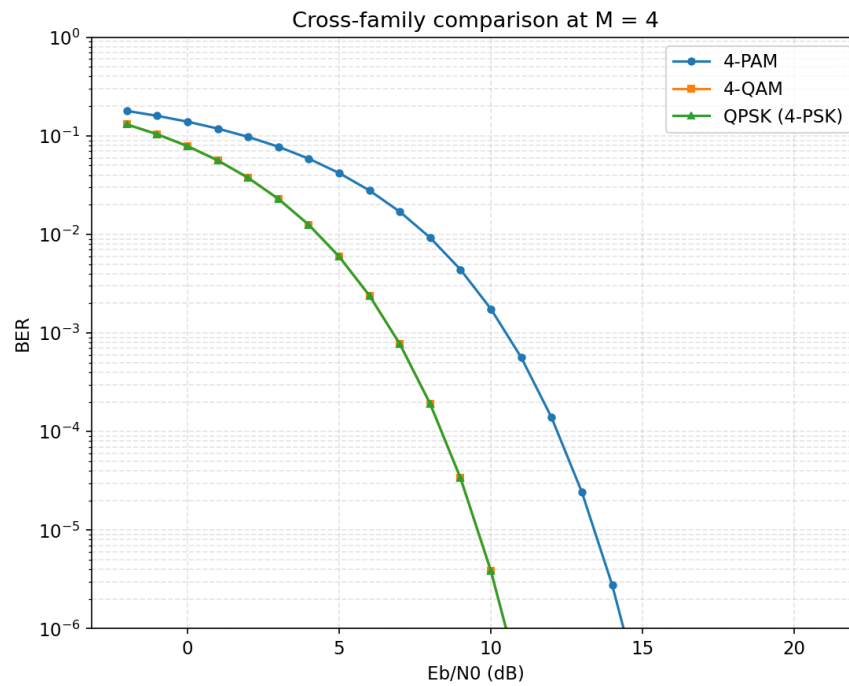


Fig. 6.4: BER vs E_b/N_0 for 4-PAM, 4-QAM and QPSK (4-PSK)

6.5 Required E_b/N_0 for a Target BER of 10^{-3}

The approximate E_b/N_0 (in dB) needed to reach a bit-error rate of 10^{-3} , read from the theoretical curves, is tabulated below for the lowest- and highest-order member of each family:

Modulation Family	M = smallest	E_b/N_0 (dB) @ $BER \approx 1e-3$	M = largest	E_b/N_0 (dB) @ $BER \approx 1e-3$
PAM	2-PAM	6.79	16-PAM	19.38
QAM	4-QAM	6.79	64-QAM	14.77
PSK	BPSK	6.79	16-PSK	14.35

7. Results & Discussion

Across all three families, the BER curves fall monotonically as E_b/N_0 increases, confirming the expected waterfall behaviour of coherent detection in AWGN. However, for a fixed E_b/N_0 , BER consistently increases with the modulation order M within each family, because packing more points into the same average-energy constellation reduces the minimum distance between neighbouring symbols and therefore increases the probability of the receiver choosing an adjacent, incorrect symbol.

Comparing families at the same order (Section 6.4, $M = 4$): QPSK and 4-QAM are identical constellations in this special case (four points on a unit circle correspond to a square grid) and their curves coincide, both outperforming 4-PAM. This illustrates that spreading the same number of points over two dimensions (I and Q) rather than a single axis increases the minimum distance for a given average energy, giving 2-D schemes (QAM, PSK) a power-efficiency advantage over 1-D PAM of the same order.

Within PSK, moving from BPSK/QPSK to 8-PSK and 16-PSK costs roughly 3–5 dB and 7–8 dB respectively at $BER = 10^{-3}$ (Table in Sec. 6.5), because all M -PSK points share the same energy (a circle) and only the angular spacing shrinks — a geometry that degrades quickly with M . QAM instead spends the extra points on both amplitude and phase, so for the same order (e.g., 16-QAM vs 16-PSK) QAM requires noticeably less E_b/N_0 for the same BER (10.5 dB vs 14.4 dB), confirming QAM's superior power efficiency at higher orders — which is why QAM (not PSK) is preferred in modern high-throughput systems (Wi-Fi, LTE/5G, DOCSIS, etc.) once M exceeds about 8.

PAM remains the least power-efficient of the three at equal M because it uses only one dimension; it is nonetheless attractive in baseband/wireline systems (e.g., Ethernet PAM-4) where a single real-valued channel is available and 2-D modulation is not applicable.

It is important to note that all curves plotted here are theoretical/analytical expressions (not Monte-Carlo simulation results); the M -PSK expression for $M \geq 8$ and the 8-QAM expression are close approximations rather than exact closed forms, but they match simulated performance very closely at the moderate-to-high E_b/N_0 values of interest.

8. Conclusion

The BER-vs-SNR and constellation analysis carried out in this experiment confirms the fundamental trade-off in digital modulation between spectral efficiency (bits/symbol, i.e., data rate) and power efficiency (noise robustness):

- Within every family (PAM, QAM, PSK), increasing the modulation order M increases the achievable data rate but requires a higher E_b/N_0 to sustain the same BER, because the minimum distance between constellation points shrinks.
- Two-dimensional schemes (QAM, PSK) outperform one-dimensional PAM of the same order because they use both the I and Q axes to separate symbols.
- For $M \geq 8$, QAM outperforms PSK of the same order because QAM varies both amplitude and phase, whereas PSK is confined to a single-radius circle, which becomes increasingly crowded as M grows.
- BPSK and QPSK provide identical per-bit error performance, so QPSK is generally preferred as it doubles the throughput for the same bandwidth and power.

These results explain common system-design choices: robust low-order schemes (BPSK/QPSK, 4-QAM) are used under poor channel conditions, while higher-order QAM (16/64/256-QAM) is switched in adaptively when the SNR is high, in order to maximise throughput without exceeding an acceptable BER.